

Aula 10 – Comunicação serial

O periférico que gera o protocolo sozinho

Microcontroladores – DENE/UFMT

OBJETIVOS

- Distinguir um protocolo gerado por software de um gerado por periférico dedicado, e quantificar a diferença em ciclos bloqueados.
- Descrever o quadro assíncrono e explicar por que transmissor e receptor precisam concordar previamente com a taxa.
- Derivar o valor do divisor de taxa a partir de F_{osc} , calcular o erro resultante e julgá-lo contra o orçamento de erro do quadro.
- Diagnosticar uma ligação serial pela aparência do defeito, distinguindo tela em branco de texto corrompido.
- Implementar transmissão por interrupção com fila circular, e justificar por que a versão bloqueante é pior que o display.
- Decidir quando usar uma biblioteca pronta, sabendo quais escolhas ela toma no seu lugar.

1 O display mostra o instante

O encontro 9 terminou com duas grandezas para medir: a amplitude da oscilação e o número de acionamentos por hora. Nenhuma das duas cabe numa tela de dezesseis colunas.

Para contar acionamentos por hora é preciso observar por uma hora. Para medir amplitude é preciso registrar o valor ao longo do tempo. O display mostra o agora; ele não guarda a história.

Este é o momento em que o sistema deixa de ser autossuficiente. Até aqui tudo que importava acontecia dentro da placa. A partir de agora existe um segundo computador na história, e o problema passa a ser **como os dois conversam**.

CONCEITO

2 Quem gera o protocolo

O encontro 3 fez uma promessa: a distinção que importa não é o número de fios, é quem produz as bordas. Aqui ela se paga.

	Display HD44780	EUSART
Quem move as linhas	O seu código, uma borda por instrução	O periférico, sozinho
O que o programa faz	Escreve LAT, espera, escreve, espera	Escreve um byte em TXREG e vai embora
Custo por byte	Cerca de 160 ciclos bloqueados	Alguns ciclos, e o hardware leva 1,04 ms sem o processador
Se o programa demorar	Nada acontece — o display espera	Nada acontece — o byte já está a caminho

Note a assimetria da última linha. No display, o tempo de transmissão é tempo de processador. Na serial, os dois relógios correm em paralelo: o byte leva um milissegundo para sair do fio, e durante esse milissegundo o processador faz outra coisa.

CONCEITO

É a primeira vez neste curso que uma tarefa acontece **simultaneamente** à execução do programa. O temporizador do encontro 5 já contava sozinho; agora um periférico inteiro trabalha sozinho.

3 O quadro

Dois fios, e nenhum deles carrega relógio. Isso obriga um acordo prévio.

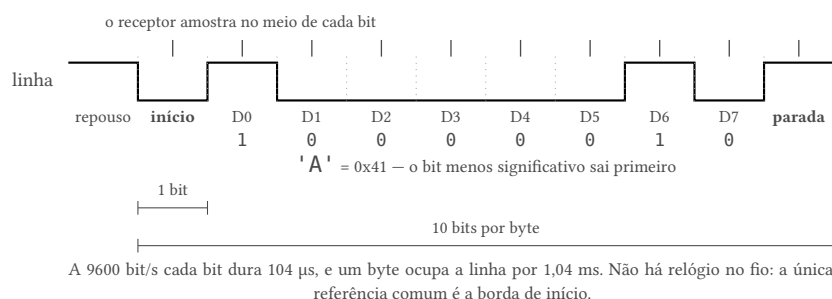


Figura 1: O quadro de dez bits. A borda de início é a única referência de tempo comum entre os dois lados, e a partir dela o receptor conta sozinho.

A linha em repouso fica em nível alto. A transmissão começa com um bit de início em nível baixo — é essa borda de descida que o receptor detecta.

CONCEITO

A partir dela ele **conta**: espera meio bit para cair no centro do primeiro bit de dado, e depois amostra a cada bit. Os oito bits de dado saem do menos significativo para o mais significativo. Um bit de parada em nível alto fecha o quadro e deixa a linha pronta para a próxima borda de descida.

Dez tempos de bit por byte, dos quais oito carregam informação: 80% de eficiência, e é o preço de não ter fio de relógio.

3.1 O divisor, e por que a taxa quase nunca é exata

A taxa não é programada em bits por segundo: é programada como um divisor inteiro do oscilador. A taxa que sai é consequência.

Com o gerador de dezesseis bits em alta velocidade, a relação é

CONCEITO

$$\text{taxa} = F_{\text{osc}} / (4(n + 1))$$

Para 9600 bit/s a 16 MHz:

$$n = 16 \cdot 10^6 / (4 \cdot 9600) - 1 = 415,67$$

O divisor é inteiro, e 415,67 não é. Com $n = 416$:

$$\text{taxa real} = 16 \cdot 10^6 / (4 \cdot 417) = 9592,3 \text{ bit/s}$$

$$\text{erro} = -0,08\%$$

Taxa desejada	n	Taxa real	Erro	Veredito
9600	416	9592,3	- 0,08%	Ótimo
9600 (8 bits, BRGH)	103	9615,4	+ 0,16%	Também serve
115200	34	114285,7	- 0,79%	Serve, com menos margem
115200 ($n = 33$)	33	117647	+ 2,1%	Arriscado

Qual erro é aceitável. O receptor amostra o bit de parada 9,5 tempos de bit depois da borda de início. Se o desvio acumulado ali passar de meio tempo de bit, a amostra cai no bit errado:

CONCEITO

$$0,5 / 9,5 \approx 5,3\%$$

Esse é o limite teórico do enlace **inteiro**, e ele precisa acomodar os dois lados. Como o computador do outro lado também tem o erro dele, a regra prática é manter cada lado abaixo de 2%.

Os 2,1% da última linha da tabela não estão errados por si — estão errados porque não sobra margem para o parceiro.

3.2 Diagnóstico pela aparência do defeito

Duas falhas comuns produzem sintomas **diferentes**, e a diferença identifica a causa sem osciloscópio.

CONCEITO

Tela em branco, nada aparece. O programa do computador recusou a taxa. Isso acontece quando você calcula um divisor que produz uma taxa fora da lista padrão — 9592 é aceito como “9600”, mas 7350 não é aceito como nada. O driver não reclama; ele simplesmente não abre.

Texto corrompido, mas com estrutura periódica. A taxa foi aceita pelos dois lados e está errada. Os quadros são lidos, mas em posições deslocadas, e o padrão do lixo se repete porque o erro é sistemático.

Branco significa “não combinamos”; lixo periódico significa “combinamos o número errado”.

4 Níveis e o que é USB

O pino do microcontrolador troca entre 0 e 5 V. Isso não é RS-232, que usa tensões de sinal invertidas e da ordem de ± 12 V, e não é USB, que não é uma porta serial de forma alguma.

A placa traz um conversor entre o nível do pino e o que o computador espera. Sem ele, ligar o pino direto num conector DE-9 de RS-232 aplicaria – 12 V num pino que suporta – 0,3 V — e o encontro 8 já explicou o que os diodos de proteção fazem nessa situação.

NESTE KIT

Verificar antes do R10: qual conversor a placa usa, e em quais pinos o RX e o TX do microcontrolador estão ligados.

4.1 Vinte minutos sobre USB

Vocês usam USB toda semana, no bootloader, sem que ninguém tenha dito o que é.

CONCEITO

USB não é uma porta serial. É um barramento com um lado mestre — o computador — que endereça dispositivos, organiza o tráfego em pacotes com verificação de erro, e faz **enumeração**: ao conectar, o dispositivo se descreve, e o computador decide qual driver carregar a partir dessa descrição.

O “COM virtual” que aparece no computador é uma **emulação**. Existe uma classe de dispositivo cuja descrição diz “eu me comporto como uma porta serial”, e o sistema operacional cria a porta. Não há UART nenhuma envolvida.

O bootloader do kit usa outra classe, a de dispositivos de interface humana — a mesma de teclados e mouses. O motivo é prático: essa classe tem driver embutido em todo sistema operacional, e por isso o bootloader funciona sem instalar nada.

E aqui fecha um número que ficou solto no encontro 2. A tabela de frequências daquela aula dizia que 48 MHz é “o que o módulo USB exige, e não o que o núcleo usa”.

NOTA

Agora dá para dizer por quê: a temporização do barramento USB é fixa pela especificação, e o módulo precisa de um relógio que a produza exatamente. O núcleo roda a 16 MHz porque essa parte não tem nada a ver com a outra.

5 Ligar o periférico

```
void uart_iniciar(void)
{
    TRISCBits.TRISC6 = 0;    /* TX: saída */
    TRISCBits.TRISC7 = 1;    /* RX: entrada */
}
```

```

    BAUDCONbits.BRG16 = 1;    /* gerador de 16 bits          */
    TXSTAbits.BRGH      = 1;    /* alta velocidade -> divisor 4 */
    SPBRGH = 0x01;           /* 416 = 0x01A0                */
    SPBRG  = 0xA0;

    TXSTAbits.SYNC = 0;        /* assíncrono */
    RCSTAbits.SPEN = 1;        /* liga o periférico, assume os dois pinos */
    TXSTAbits.TXEN = 1;        /* habilita transmissão */
    RCSTAbits.CREN = 1;        /* habilita recepção */
}

```

Os dois bytes de SPBRGH: SPBRG são o 416 da seção anterior, e é a única linha deste bloco que exigiu conta. Todo o resto são chaves de configuração: síncrono ou assíncrono, oito ou dezesseis bits de divisor, alta ou baixa velocidade.

NOTA

Vale reparar em SPEN: ligá-lo entrega os dois pinos ao periférico. A partir daí LATC6 não move mais nada — o encontro 2 dizia que escrever num endereço move elétrons num fio, e esta é a primeira exceção que o curso encontra. O fio passou a pertencer a outro dono dentro do próprio chip.

6 Enviar sem bloquear

Escrever um byte e esperar que ele saia é a primeira ideia, e é a pior de todas as versões que este curso já viu.

```

void uart_byte(uint8_t b)
{
    while (!PIR1bits.TXIF);    /* espera a fila esvaziar */
    TXREG = b;
}

```

A 9600 bit/s cada byte ocupa a linha por 1,04 ms. Uma linha de telemetria com 25 caracteres bloqueia o processador por **26 ms**.

Compare: a atualização de tela do encontro 3 bloqueava por 1,3 ms e já era o pior do sistema. A transmissão bloqueante é **vinte vezes pior**, e passa por cima do meio período de 1,14 ms da nota, do intervalo de 5 ms do filtro de repique e de qualquer pretensão de controle em tempo real.

Mesmo a 115200 são 2,17 ms — ainda pior que o display.

A saída é a do encontro 7. O programa deposita a linha numa fila circular e segue. Quando o transmissor esvazia, ele pede uma interrupção; o tratamento retira um byte da fila e o entrega ao periférico.

CONCEITO

Custo: algumas dezenas de ciclos por byte, espalhados. Com uma linha por segundo, menos de 0,1% do processador — e **zero** bloqueio.

Este é o padrão que aparece em todo sistema embarcado que transmite algo, e é a terceira vez neste curso que a mesma lição se apresenta: quem espera desperdiça, quem é interrompido não.

```

#define FILA 64
static volatile uint8_t fila[FILA];
static volatile uint8_t entra = 0, sai = 0;

void uart_enfileirar(uint8_t b)
{
    uint8_t prox = (uint8_t)((entra + 1u) % FILA);
    while (prox == sai) { }    /* fila cheia: raro, e aqui sim espera */
}

```

```

    fila[entra] = b;
    entra = prox;
    PIE1bits.TXIE = 1;                /* acorda o transmissor */
}

/* dentro do tratamento de alta prioridade */
if (PIE1bits.TXIE && PIR1bits.TXIF) {
    if (entra == sai) {
        PIE1bits.TXIE = 0;          /* nada a enviar: desliga a fonte */
    } else {
        TXREG = fila[sai];
        sai = (uint8_t)((sai + 1u) % FILA);
    }
}
}

```

Duas escolhas que valem explicação.

NOTA

TXIE é ligado ao enfileirar e desligado quando a fila esvazia. Se ficasse sempre ligado, o transmissor pediria interrupção continuamente com a fila vazia — um laço infinito disfarçado de sistema ocioso.

A espera na fila cheia é a única espera que sobrou, e ela só acontece se o programa produzir dados mais rápido do que o enlace escoar. Se isso for comum, o problema não é a fila: é a taxa de geração.

7 Formatar sem gastar a Flash

Montar a linha 125300,398,400,1,17 exige converter números em texto. O caminho curto existe e tem preço.

```

/* Redireciona printf para a fila da serial. Basta definir putchar. */
void putchar(char c)
{
    uart_enfileirar((uint8_t) c);
}

/* ... e a partir daqui: */
printf("%lu,%d,%d,%u,%u\r\n", ms, temp_d, alvo_d, aquecedor, comutacoes);

```

printf é uma linha de código e da ordem de **1 a 2 kB de Flash** — algo entre 3% e 6% dos 32 kB do chip, só para o formatador inteiro. Com ponto flutuante seria muito mais, e o compilador na versão gratuita não oferece essa opção de qualquer forma (encontro 4).

ATENÇÃO

O número exato aparece no arquivo de mapa gerado pela compilação. **Meça o seu**, antes e depois de incluir printf, em vez de acreditar nesta estimativa.

A alternativa custa algumas dezenas de bytes:

CONCEITO

```

/* Escreve um inteiro sem sinal na fila, sem printf. */
static void enviar_u16(uint16_t v)
{
    char buf[6];
    uint8_t i = 0;

    do {
        buf[i++] = (char)('0' + (v % 10u));
        v /= 10u;
    } while (v != 0u);

    while (i != 0u) {

```

```

        uart_enfileirar((uint8_t) buf[--i]);    /* na ordem inversa */
    }
}

```

O veredito honesto: use `printf` enquanto desenvolve, meça o mapa, e troque só se o espaço apertar. Otimizar antes de medir é o erro que o encontro 3 já cobrou uma vez.

8 Receber

Transmitir resolve a telemetria. Receber resolve o outro sentido — ajustar o alvo pelo computador, sem mexer no botão.

```

volatile char linha[24];
volatile uint8_t n = 0;
volatile uint8_t linha_pronta = 0;

/* dentro do tratamento */
if (PIR1bits.RCIF) {
    if (RCSTAbits.OERR) {          /* estouro: a recepcao PAROU */
        RCSTAbits.CREN = 0;
        RCSTAbits.CREN = 1;      /* religar e a unica saida */
    }

    char c = RCREG;                /* ler RCREG limpa RCIF */

    if (c == '\n' || c == '\r') {
        linha[n] = '\0';
        linha_pronta = 1;
        n = 0;
    } else if (n < sizeof(linha) - 1) {
        linha[n++] = c;
    }
}
}

```

O estouro de recepção é o defeito mais confuso desta aula.

ATENÇÃO

O receptor guarda dois bytes enquanto o programa não os retira. Se um terceiro chegar antes, o indicador `OERR` sobe — e, a partir daí, **a recepção para permanentemente**. Não é uma perda de um byte: é o fim da recepção até que alguém desligue e religue `CREN`.

O sintoma na bancada: a serial funciona por dois minutos e depois emudece, sem travar o programa, sem reiniciar nada. A transmissão continua normal, o que aponta o dedo para o cabo, para o computador e para tudo menos para a causa.

A causa costuma ser um tratamento que demorou demais — por exemplo, porque alguém escreveu no display de dentro dele.

Ler `RCREG` é o que limpa `RCIF`. Um tratamento que testa `RCIF` e não lê `RCREG` volta a ser chamado imediatamente, para sempre. É o mesmo modo de falha do `TXIE` sempre ligado, por outro caminho.

NOTA

9 E as bibliotecas?

Sim, existem, e neste ponto do curso vale usá-las.

Opção	O que entrega	O que decide por você
Configurador gráfico da Microchip	<code>uart_iniciar</code> , escrita e leitura, com ou sem fila	O divisor, o tamanho da fila, e se a escrita bloqueia

Bibliotecas de periférico antigas do compilador	OpenUSART, putsUSART	O mesmo, e estão descontinua-
printf com putch	Toda a formatação	1 a 2 kB de Flash
Serial do Arduino	Tudo, inclusive a fila	Absolutamente tudo

A posição deste curso não é “não use biblioteca”. É: derive uma vez, e depois use para sempre.

CONCEITO

O motivo de calcular o 416 à mão é que, quando o configurador mostrar uma caixa escrita “9600”, você saiba que ele programou 9592,3, por que programou isso, e que o erro de 0,08% cabe no orçamento. Quem nunca fez a conta não tem como julgar a caixa.

E há uma escolha que a biblioteca toma silenciosamente e que decide o comportamento do seu sistema: **se a escrita bloqueia ou enfileira**. As duas opções existem no mesmo configurador, atrás de uma caixa de seleção — e a diferença entre elas são os 26 ms desta aula.

Regra prática para o resto do curso, e para depois dele: se você não sabe dizer o que a biblioteca faz nos três casos abaixo, ainda não pode usá-la sem ler.

NOTA

Quando a fila enche. Quando um byte chega e ninguém leu o anterior. Quando o periférico está ocupado e você chama a função de escrita.

10 O que enviar

Para o encontro 13 o formato importa, porque esses dados viram um conjunto de treino em Python.

	Texto separado por vírgulas	Binário compacto
Tamanho de uma amostra	cerca de 25 bytes	8 bytes
Tempo de linha a 9600	26 ms	8,3 ms
Ler no terminal	Direto	Impossível
Ler em Python	Uma linha de código	Precisa de struct e do formato certo
Sobrevive a um byte perdido	Sim — perde-se uma linha	Não — desalinha tudo

Para este projeto, texto. A telemetria é de uma amostra por segundo, e 26 ms sobre 1000 são 2,6% do enlace: sobra folga de quarenta vezes.

CONCEITO

Binário se justifica quando a banda é o recurso escasso. Aqui o recurso escasso é o tempo de quem depura, e texto é imbatível nesse critério.

```
t_ms,temp_d,alvo_d,aquecedor,comutacoes
125300,398,400,1,17
126300,401,400,0,18
```

11 Previsão para o R10

PREVISÃO

P1. Com $n = 416$, que taxa real você espera medir no osciloscópio? Qual é a largura de um bit?

P2. Você vai abrir o terminal com a taxa configurada em 19200 em vez de 9600. O que espera ver — nada, ou lixo? Justifique com o que a aula disse.

P3. Envie uma linha de telemetria por segundo com a versão bloqueante e ouça o buzzer tocando junto. Descreva o que espera ouvir.

P4. Troque para a versão com fila e interrupção. O que muda no som? E no osciloscópio, no pino TX?

P5. Quantas horas de telemetria a 1 Hz cabem num arquivo de 1 MB? Faça a conta com o formato de texto da aula.

Leve esta folha preenchida. O arquivo gerado neste roteiro é a matéria-prima do encontro 13 – guarde-o.

SEM NOTA

12 Exercícios

Exercício 10.1. Um enlace a 9600 funciona entre a placa e o computador, mas falha com um segundo dispositivo, que tem erro de taxa de + 1,8%.

TAREFA

(a) Qual é o erro total do enlace, no pior caso?

(b) Ele cabe no orçamento?

(c) O defeito vai aparecer como tela em branco ou como texto corrompido?

Exercício 10.2. Compare o custo de bloqueio das três coisas que o termostato faz, a 16 MHz, e ordene da pior para a melhor.

TAREFA

(a) Atualizar a tela.

(b) Enviar uma linha de telemetria de 25 bytes a 9600, com a versão bloqueante.

(c) Ler o conversor analógico.

Exercício 10.3. Explique por que TXIE precisa ser desligado quando a fila esvazia, e o que acontece se ele ficar sempre ligado.

TAREFA

Exercício 10.4. O termostato precisa transmitir uma amostra por segundo durante oito horas para o conjunto de dados do encontro 13.

TAREFA

(a) Quantas linhas?

(b) Qual o tamanho do arquivo, no formato de texto da aula?

(c) Qual fração do enlace a 9600 é usada?

(d) Se você quisesse dez amostras por segundo, o formato de texto ainda serviria?

Exercício 10.5. A telemetria funciona, mas a recepção de comandos para de responder depois de alguns minutos. A transmissão continua normal e o programa não trava.

TAREFA

(a) Qual indicador você inspecionaria primeiro?

(b) Por que o defeito é permanente, e não intermitente?

(c) Cite duas causas plausíveis, e diga qual delas o encontro 3 já anunciava.

No encontro 11: barramento serial síncrono e memória não volátil. A telemetria resolve o registro **fora** da placa, e deixa um problema dentro dela: o alvo que o usuário ajustou desaparece no próximo reset.

NOTA

E o encontro 8 já mostrou que reset, neste projeto, não é um evento raro.